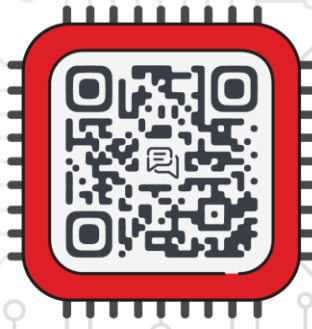




نظري

7

13

1950 sp

كلية الهندسة المعلوماتية

السنة الرابعة - قسم البرمجيات

Project Management & Project Planning

د. سعيد أبو طراب



محتوى مجاني غير مخصص للبيع التجاري
RB Informatics; 13/05/2026

هندسة برمجيات ٢

Software Project Management

هل مجرد أن الكود يعمل يعني أن المشروع ناجح فعلاً؟

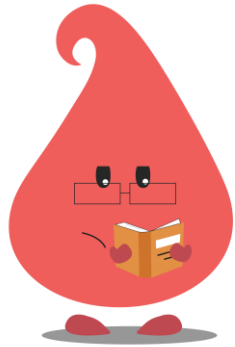
هنا تبدأ هندسة البرمجيات بالخروج من فكرة "الكود فقط" إلى فكرة "إدارة العمل كاملاً".

لأن أي مشروع برمجي في النهاية ليس مجرد ملفات وكلاسات وخوارزميات، بل مجموعة أشخاص يعملون معاً للوصول إلى هدف معين.

ولهذا، بعد الانتهاء من:

Main Process Models سنبدأ بأول topic في Umbrella Activity وهي Project Management

ما هي عوامل النجاح (Success Factors) ؟



هل تم تسليم المشروع بالوقت المحدد؟

هل تم تحقيق متطلبات العميل؟

هل بقي المشروع ضمن الميزانية (Budget) ؟

هل حافظنا على استقرار الفريق؟

لأنك أحياناً قد تنجح في إنهاء المشروع، لكن بعد ذلك يغادر الفريق بالكامل بسبب الضغط وسوء الإدارة.

هل توجد طريقة قياسية وثابتة للإدارة؟

لا، بل تعتمد على عدة عوامل، أهمها:

1. حجم الشركة (Company Size)

هناك فرق كبير بين:

شركة فيها 10 موظفين

شركة فيها 1000 موظف

كلما كبر حجم الشركة زادت الحاجة إلى:

التوثيق

المتابعة

التنظيم

الإجراءات

2. حجم المنتج البرمجي (Product Size)

يوجد فرق بين:

- Enterprise Application
- تطبيق بسيط تم تطويره خلال أسبوعين
- فكل مشروع يفرض متطلبات مختلفة للإدارة.

3. مستوى نضج الفريق (Team Maturity Level)

إذا كان لدى فريق:

- يمتلك خبرة
- يفهم المطلوب
- يعمل باستقلالية

فإن طريقة إدارته تختلف عن فريق ما يزال في مرحلة التعلم.
لكن تبقى نقطة شديدة الأهمية: الثقافة (Culture) لأن الثقافة هي التي تحدد طريقة العمل داخل الفريق.

- الإدارة تعتمد على أربعة أبعاد رئيسية تبدأ جميعها بالحرف P:

Project

Product

People

Process

- كل مشروع (Project) يمتلك إطاراً خاصاً به.
- لكي نستطيع إدارة هذا المشروع بالشكل الصحيح، نحتاج إلى عملية أو إجراء (Process) ذكرنا سابقاً، يوجد ما يسمى Process Model :
جزء كبير من الإدارة يتمثل في اختيار النموذج الإجرائي المناسب والتأكد من تطبيقه بالشكل الصحيح، فاختيار المنهجية وحده لا يكفي، بل يجب التأكد من أن طريقة العمل نفسها تُنفذ كما هو مخطط لها.
- المخرج الرئيسي للمشروع هو: المنتج (Product)

ولذلك يجب تحديد:

- Features
- احتياجات المستخدم
- الخصائص المطلوبة
- توقعات العميل
- الذي ينفذ هم الأشخاص (People) ، ولهذا تُعد إدارة الأشخاص من أهم أجزاء الإدارة البرمجية.

المحور الأول: People Management إدارة الأشخاص

نجاح المشروع لا يعتمد فقط على التكنولوجيا أو الأدوات، بل يعتمد بدرجة كبيرة على الفريق نفسه.
فالقيمة الحقيقية داخل شركات البرمجيات ليست في الأجهزة أو المكاتب، وإنما في الأشخاص الذين يبنون المنتج فعلياً ويعملون ضمن فريق متكامل ومنسجم.
وإذا استطاعت الإدارة بناء هذا النوع من الفرق، فإنها تكون قد حققت جزءاً كبيراً من النجاح في إدارة المشروع.

ما هي عوامل إدارة الأشخاص؟ ما الذي يجعلك تتقبل مديراً معيناً وترفض مديراً آخر؟

1. العامل الأول: الاحترام (Respect)

الإنسان بطبيعته يريد أن يشعر بقيمته.

2. العامل الثاني: المساواة (Equality)

التعامل بعدالة وعدم التمييز بين أعضاء الفريق.

تخيل وجود حالات مثل:

- شخص يحصل دائماً على معاملة خاصة
- شخص يتحمل معظم العمل
- شخص كثير الغياب ولا تتم محاسبته

عندها يبدأ الشعور:

- بالظلم
- بعدم التقدير
- بفقدان الحماس

ولهذا فإن العدالة في:

1. توزيع المهام
2. التقدير
3. المكافآت
4. المتابعة

تعد عاملاً أساسياً جداً.



3. العامل الثالث: تقدير الأفكار

شعور الشخص بأن رأيه مسموع يجعله أكثر انتماءً وحماساً للعمل.

ما الذي يجعلك تثق بالمدير؟

إذا وعدك مثلاً:



بفرصة تطوير



بمكافأة



بترقية

فما الذي يجعلك تصدق هذا الوعد؟

هنا تظهر نقطة مهمة جداً: الثقة (Trust) الثقة بالشركة، بالمدير وبأن الوعود ستنفذ فعلاً.

وعندما تشعر أن الشخص الذي يقود المشروع:

- منظم
- يوزع المهام جيداً
- يضع الشخص المناسب في المكان المناسب
- يتابع العمل بوضوح

فإن ذلك ينعكس مباشرة على شعورك تجاه العمل خصوصاً عندما توجد:

- مهام واضحة
- مسؤوليات واضحة
- تقديرات زمنية واضحة
- خطط لتطوير المهارات

عندها يبدأ الشخص بالشعور أنه لا يعمل فقط، بل يتطور أيضاً.

❖ ماذا تعني عملية ال Motivation؟

يُعد تحفيز الموظفين الدور الجوهري لمدير المشروع، وهو يتمثل في استخراج أفضل ما يملكه الموظف، فكلما كانت عملية ال Motivation عالية كانت ال Activities عالية. وبالمقابل فإن غياب التحفيز يؤدي لبطء العمل، زيادة الأخطاء، وعدم الاكتراث بأهداف المنظمة.

هناك أنواع مختلفة لل Motivation تركز على:

1. Basic needs
2. Personal needs
3. Social needs

ويمكن تصميم الاحتياجات البشرية وفقاً للهرمية التالية:

- Physiological needs: تتعلق بالمتطلبات البيئية الأساسية التي تضمن راحة الموظف الجسدية والنفسية أثناء العمل.
- Safety needs: تتمثل في شعور الفرد بالأمان والاستقرار داخل المنظمة ليتمكن من أداء عمله بشكل سليم.
- Social needs: بما أن الإنسان اجتماعي بطبعه، فهو بحاجة للانتماء والانسجام مع الفريق، وتوفير مرافق مشتركة (Communal facilities) للتواصل العفوي.
- Esteem needs: تتعلق بتقدير الذات والحصول على اعتراف من الآخرين بالقيمة المهنية، وتقدير المدير لنجاحات الموظفين.
- Self-realization needs: قمة الهرم، وتتعلم بالنمو الشخصي والشعور بأن العمل له قيمة وتأثير حقيقي.

أنواع الشخصيات:

تعد معرفة أنواع الشخصيات وفهم دوافعها أمراً حيوياً لمدير المشروع الناجح، حيث يساعد ذلك في وضع الشخص المناسب في المكان المناسب وتشكيل فريق متناعم وقادر على الإنتاج، ونميز ثلاث شخصيات:

1. الموجهون نحو المهمة (Task-oriented):

هؤلاء الأشخاص يحركهم ويحفّزهم نوع العمل بحد ذاته، كما يميلون غالباً للعمل بشكل منفرد، وهم قادرون على إنجاز عدد كبير من المهام بكفاءة عالية طالما أنها واضحة ومحددة، لكنهم قد يجدون صعوبة في ابتكار مهام جديدة من تلقاء أنفسهم أو الخروج عن النطاق التقني الذي يبرعون فيه.

2. الموجهون نحو التفاعل (Interaction-oriented) :

المحفز الأساسي لهؤلاء هو وجود الزملاء والتواصل معهم؛ فهم يذهبون للعمل لأنهم يحبون بيئة العمل الجماعية، فهم لا يفضلون العمل بمفردهم، هم "صمام الأمان" في الفريق؛ فلهم قدرة عالية على اكتشاف التوترات وحلها قبل أن تتفاقم، كما يفضلون اتخاذ القرارات بالتشاور مع الآخرين، ويؤمنون بأن الحصول على المعلومة من مصادر متعددة داخل الفريق يجعلها أكثر دقة وفائدة.

3. الموجهون نحو الذات (Self-oriented) :

يحفزهم دافع داخلي وشخصي يتمثل في النجاح الفردي والتميز المهني، فهم أشخاص منظمون جداً لأفكارهم ويلتزمون بالعمل الجاد طالما أنه يخدم مسارهم المهني وتطورهم الشخصي.

ملاحظة: يجب على مدير المشروع تحقيق التوازن من خلال اختيار خليط من هذه الشخصيات؛ فالفريق الفعّال هو الذي يستثمر في تنوع المهارات وطرق التفكير لتعويض نقص الأفراد ببعضهم البعض.

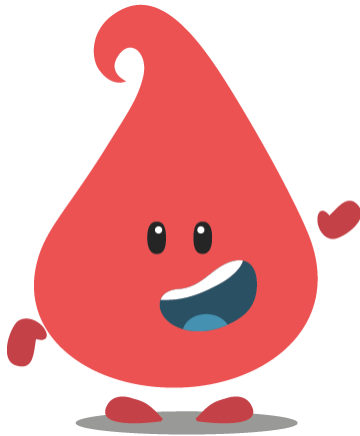
❗ إن أساس عمل ال software يعتمد على ال Team، هناك قاعدتين لبناء Team صحيح:

اختيار أعضاء ال team من شخصيات مختلفة (تنوع الشخصيات)

وجود Team Leader لديه مهارات تنظيمية وقدرة على توزيع ال tasks بشكل صحيح

تعتمد عملية اختيار الفريق على:

- صعوبة المشكلة المراد حلها.
- حجم الحل.
- المدة التي سيبقى فيها الفريق معاً (عمر الفريق).
- مدى إمكانية تقسيم المشكلة إلى وحدات.
- الجودة المطلوبة وموثوقية النظام المراد بناؤه.
- صرامة موعد التسليم.
- درجة التواصل المطلوبة للمشروع.



المحور الثاني: (Software Team Organization)

يختلف تنظيم الفريق حسب حجم المشروع وصعوبته، وهناك أربعة نماذج تنظيمية:

1. النموذج المغلق (Closed Paradigm)

يعتمد هذا النموذج على **الميكالية الهرمية التقليدية** للسلطة، حيث تكون الصلاحيات واضحة والقرارات مركزية.

آلية العمل :

إذا أراد موظف اتخاذ قرار، يجب عليه إبلاغ قائد الفريق (Leader)، الذي يرفعه لمدير المشروع (Project Manager)، وصولاً إلى المدير التنفيذي (CEO) ومجلس الإدارة للمصادقة عليه.

الإيجابيات:

- **مركزية القرار:** تتيح رؤية شاملة لكافة موارد الشركة من "رأس الهرم"، مما يقلل من احتمالية وقوع الأخطاء.
- **الانضباط:** تضمن سير جميع العمليات (Operations) بنفس الخطى، وهو أمر حيوي في أوقات الأزمات.

السلبات:

- **عملية اتخاذ القرار بطيئة جداً** بسبب كثرة المستويات الإدارية، كما أنها قد "تقتل الإبداع" نتيجة الصرامة الشديدة.

متى يُستخدم؟

هو النموذج الأنجح في إدارة الأزمات والقرارات الحاسمة.

2. النموذج العشوائي (Random)

هو النقيض تماماً للنموذج المغلق، حيث يعتمد على **المبادرة الفردية** لأعضاء الفريق دون هيكلية واضحة.

آلية العمل:

تعتبر الفرق عرضية (Horizontal) وكل فرد يعتبر نفسه مستقلاً؛ حيث يتخذ الفريق قراراته في الوقت وبالطريقة التي يراها مناسبة دون تواصل كبير مع بقية الفرق.

الإيجابيات:

- يحفز الابتكار والإبداع بشكل كبير ويوفر الوقت.

السلبات:

- يؤدي إلى **هدر الموارد** وضياع أموال الشركة، لأن كل فريق يعمل بميزانية منفصلة دون تنسيق مركزي يمنع التكرار.

متى يُستخدم؟

لا ينصح به أبداً في **المشاريع الضخمة**، بل يقتصر على بيئات العمل التي تتطلب ديناميكية عالية وحلولاً خارج الصندوق، فهو يصلح للشركات الصغيرة.

3. النموذج المفتوح (Open Paradigm)

يحاول هذا النموذج الجمع بين ميزات النموذجين السابقين؛ فهو يوفر **التحكم** الموجود في النموذج المغلق و**الابتكار** الموجود في النموذج العشوائي.

آلية العمل: يتم تقسيم القرارات إلى مستويين:**1. قرارات استراتيجية:**

تتخذ من "رأس الهرم" وبالتنسيق بين قادة الفرق لضمان توافقها مع أهداف الإدارة.

2. قرارات تقنية بسيطة:

تمنح الإدارة فيها صلاحيات لقائد الفريق والموظفين لاتخاذها بسرعة، مثل اختيار بيئة التطوير (IDE) أو إطار العمل (Framework) المناسب.

الهدف: تحقيق التوازن بين الانضباط التنظيمي وسرعة التنفيذ التقني.

4. النموذج المتزامن (Synchronous Paradigm)

يعتمد هذا النموذج على تقسيم المشكلة طبيعياً إلى أجزاء مستقلة تعمل عليها فرق مختلفة في وقت واحد.

آلية العمل:

يتم توزيع نفس المهام أو أجزاء متكاملة منها على مجموعات متعددة تعمل بالتوازي، مع وجود تواصل مبرمج بينها.

➤ الإيجابيات:

- تنوع الحلول: يخلق نوعاً من التنافس الإيجابي بين الفرق، مما يؤدي للوصول إلى أفضل الحلول الممكنة
- التكامل: يضمن إنتاج عمل متكامل من خلال دمج مخرجات الفرق المختلفة

ملاحظة: التواصل بين أعضاء الفريق يتم عبر وسائل رسمية (اجتماعات لاتخاذ قرارات) أو غير رسمية (لقاءات عبر الإنترنت أو نقاشات عفوية) لضمان انسجام العمل

المحور الثالث: تخطيط المشروع البرمجي (Project Planning)

التخطيط هو اتخاذ قرار مستقبلي في الزمن الراهن، وهو نوع من أنواع ال Activity التي لا توصل لمنتج برمجي لكنها تزود الإنتاجية وتخفف ضياع الوقت.

مراحل التخطيط الثلاث:

1. مرحلة الاقتراح (Proposal Stage) :

تخطيط أولي لتقديم عرض فني ومالي للزبون، دون الدخول في تفاصيل دقيقة.

2. مرحلة البدء (Project Startup Phase) :

وضع خطة مفصلة للميزانية، توزيع المهام، وتحديد آليات المراقبة.

3. التخطيط الدوري (Periodically) :

تعديل الخطة بناءً على الخبرة المكتسبة ومراقبة التقدم الفعلي.

ما يتم تحديده في مرحلة التخطيط:

تحديد المهام

الوقت اللازم لكل مهمة

الموارد (شخص، أدوات، مال) الخاصة لكل مهمة

خرج عملية التخطيط:

السعر

التكلفة

الزمن النهائي

المنتج النهائي

استراتيجيات التخطيط والمخاطر:

1. النمط الاستجابي (Reactive Mode)

هذا النمط يمثل الإدارة التقليدية التي تفتقر للحس الهندسي الاستباقي، حيث يتم التعامل مع الأمور كفعل ورد فعل.

آلية العمل :

ينتظر المدير وقوع المشكلة فعلياً (مثل استقالة مبرمج مفاجئة أو اكتشاف خطأ برمجي كارثي قبل التسليم بيوم) ثم يبدأ بالبحث عن حل.

المخاطر :

ردود الفعل في هذا النمط غالباً ما تكون غير منطقية أو مشوبة بالتوتر، مما قد يؤدي لنتائج عكسية.

المنظور الهندسي:

لا يُنصح به إطلاقاً، فالفرق الجوهرى بين المهندس والشخص العادي هو قدرة المهندس على حساب كافة العوامل التي قد تؤثر على الخطة مسبقاً.

2. النمط الاستباقي (Pro-active Mode)

هو الجوهر الحقيقي لعالم دراسة المخاطر، ويهدف لتجهيز بيئة العمل لما قبل وقوع الخطر لضمان أن يكون تأثيره في أدنى مستوياته.

- تجهيز الـ Package : يقوم الخبراء بدراسة احتمالات المشاكل ووضع "حزمة (Package) "جاهزة لكل خطر محتمل؛ تشمل هذه الحزمة العمليات البديلة، التدريب اللازم، والموارد الإضافية.
- المسارات البديلة: في أي مشروع، يجب أن يتضح مسار واحد على الأقل من البداية للنهاية (الخطة الأساسية)، ولكن في النمط الاستباقي يتم وضع مسارات إضافية بديلة يتم تفعيلها عند حدوث خطر معين، علماً أن كل مسار إضافي له كلفته المادية الخاصة.
- الافتراضات الواقعية: يتطلب هذا النمط وضع افتراضات واقعية وليست متفائلة جداً، مع تضمين خطط طوارئ (Contingency) لضمان عدم تعطل جدول التسليم بشكل خطير عند وقوع خلل ما.

3. هيكلية تقسيم العمل (WBS)

تعتمد هذه الاستراتيجية على مبدأ (Top-Down Approach) ، حيث يتم تفكيك الهدف الكبير إلى أجزاء أصغر حتى نصل لمستوى المهام الفردية.

■ قاعدة يومي العمل (16 ساعة)

تنص القواعد الهندسية للتخطيط على ضرورة الاستمرار في تقسيم المهام حتى نصل لمهمة يمكن لشخص واحد إنجازها في مدة أقصاها يومين عمل (أي 16 ساعة عمل فعلية).

أهمية هذا التقسيم الحقيقي:

- المراقبة اللحظية: تمكّن هذه القاعدة المدير وقائد الفريق من إجراء "فحص التقدم اليومي (Daily)" (Progress Check) فإذا كانت المهمة تستغرق 16 ساعة فقط، يمكن التأكد من انتهائها بنهاية اليوم الثاني عبر المخرجات (Output) الفعلية.
- تخفيف المخاطر: كلما كانت المهمة أصغر، قل احتمال حدوث انحراف كبير في الجدول الزمني الكلي للمشروع عند تعثرها.
- دقة التقدير: من السهل جداً تقدير الوقت والجهد لمهمة صغيرة (مثل برمجة واجهة دخول)، بينما يصعب ذلك للمهام الكبيرة والمبهمة.
- الاستقلالية: يجب أن تكون هذه المهام الفرعية (Sub-tasks) مستقلة قدر الإمكان لسهولة إسنادها للمطورين وتجنب التبعية المعقدة التي تعطل العمل.

المحور الرابع: التخطيط في المنهجيات الرشيقة (Agile) مقابل التقليدية (Plan-Driven)

■ أولاً: (Plan-Driven Development)

في هذا النموذج، تعتبر **الخطة هي المرجع الأول والأخير** لإدارة المشروع، ويتم إعدادها بدقة متناهية قبل البدء في أي عملية تطوير فعلية.

- **التخطيط التفصيلي المسبق:** يتم تسجيل كافة المهام المراد إنجازها، وتحديد الأشخاص المسؤولين عنها، ووضع جدول زمني صارم، وتحديد مخرجات العمل (Work Products) بدقة.
- **اتخاذ القرارات المبكرة:** يميل مدراء المشاريع هنا إلى اتخاذ معظم القرارات التنظيمية والتقنية في وقت مبكر جداً، مما يساعد في كشف المشاكل والتبعيات (Dependencies) بين المهام قبل البدء، ويقلل من التكاليف التنظيمية لاحقاً.
- **أداة لقياس التقدم:** يستخدم المدير الخطة الموضوعية كمسطرة لقياس مدى تقدم الفريق، فالمعيار هو مدى الالتزام بالجدول الزمني والميزانية المحددة سلفاً.
- **مكونات خطة المشروع:** تشتمل عادة على أقسام ثابتة مثل: مقدمة الأهداف، تنظيم المشروع، تحليل المخاطر، المتطلبات العادية والبرمجية، هيكلية تقسيم العمل (WBS)، الجدول الزمني، وآليات التقرير والمراقبة.

سليباتها:

تكمن المشكلة الأساسية في صعوبة تعديل القرارات التي اتخذت مبكراً إذا ما حدثت تغييرات في بيئة العمل أو متطلبات الزبون.

ثانياً: المنهجية الرشيقية (Agile Planning)

على عكس النموذج التقليدي، يعتمد التخطيط الرشيق على التكرار (Iteration) ، حيث لا يتم التخطيط لكامل وظائف المنتج مسبقاً، بل يتم اتخاذ القرارات أثناء عملية التطوير.

- التخطيط القائم على الأولوية: يتم تحديد ما سيتم إنجازه في كل مرحلة بناءً على أولويات الزبون ومسار تقدم العمل الفعلي.

مراحل التخطيط الرشيق:

1. **تخطيط الإصدار (Release Planning)** : نظرة بعيدة الأمد (تتراوح عادة بين 4 إلى 6 أشهر) لتحديد الميزات الأساسية التي سيحتويها الإصدار النهائي للمنتج.
2. **تخطيط التكرار (Iteration Planning)** : خطة قصيرة الأمد (تتراوح عادة بين 2 إلى 4 أسابيع) تركز على المهام المحددة التي سيقوم بها الفريق في (Sprint) القادمة.

- مرونة النطاق (Scope) : وقت انتهاء المرحلة (Iteration) ثابت ولا يمكن تجاوزه، وإذا لم تكتمل المهام، يتم تضيق نطاق العمل لضمان التسليم في الوقت المحدد بدلاً من تمديد الموعد النهائي.
- توزيع المهام الذاتي: يقوم المطورون بتقسيم القصص (Stories) إلى مهام صغيرة (تستغرق كل مهمة 4-16 ساعة)، ثم يختار كل مطور المهام التي سينفذها بنفسه؛ مما يزيد من حافزيتهم وشعورهم بملكية العمل.

ثالثاً: التحديات الجوهرية في المنهجيات الرشيقية (Agile Challenges)

1. **إشراك الزبون المستمر**: يعتمد نجاح التخطيط الرشيق كلياً على تواجد الزبون وتفاعله الدائم لتحديد الأولويات، وهو أمر يصعب تحقيقه أحياناً بسبب انشغال الزبون أو عدم فهمه لهذا النمط من العمل.
2. **تغير المتطلبات الدائم**: التغيير المستمر يجعل التخطيط في حالة سيولة دائمة، مما قد يربك الفريق إذا لم يكن لديه القدرة على التكيف السريع .
3. **الحاجة لفريق مستقر (Stable Team)** : تعتمد الـ Agile بشكل كبير على **الثقة والانسجام** داخل الفريق. يجب أن يظل الفريق مستقراً لعدة شهور دون تغيير في أعضائه لضمان التفاهم في توزيع المهام (Tasks) ؛ فالتغير المتكرر في الأعضاء يقتل قدرة الفريق على التخطيط التعاوني.
4. **التحديات التنظيمية**: يصعب تطبيق هذا النمط في الفرق الكبيرة جداً أو الموزعة جغرافياً، حيث يصبح التواصل المباشر والتعاون الوثيق (الضروريان للتخطيط الرشيق) أمراً شبه مستحيل.

سنبلغ حلمنا لو بعد حين ...

فنحن بحار عزم إن أردنا



المحور الخامس: تقنيات التقدير وتسعير البرمجيات:

أولاً: تقنيات التقدير (Estimation Techniques)

تعد عملية التقدير (Estimation) من أصعب المهام في إدارة المشاريع البرمجية، حيث يجب أن تحاكي التوقعات الواقعية بدقة، مع العلم أن الحجم الحقيقي للمشروع لا يمكن معرفته بدقة تامة إلا عند الانتهاء من تطويره.

أولاً: التقدير القائم على الخبرة (Experience-Based Techniques)

تعتمد هذه التقنية بشكل أساسي على المحاكاة المنطقية المستمدة من تجارب سابقة لمدير المشروع في مشاريع أو مجالات عمل مشابهة.

آلية العمل:

1. يقوم المدير بتحديد المخرجات (Deliverables) والمكونات البرمجية المراد تطويرها.
2. يتم توثيق هذه المكونات في جداول بيانات (Spreadsheets) وتقدير جهد كل منها على حدة، ثم حساب الجهد الكلي المطلوب.
3. تعتمد هذه الطريقة على التوقع بالقياس، أي مقارنة المشروع الحالي بمشاريع تشبهه تم إنجازها سابقاً، ولا تتم هذه المقارنة إلا بعد فهم عميق لنطاق المشروع الحالي.

ملاحظة: من المفيد إشراك أكثر من شخص في عملية التقدير لمشاركة الخبرات ومناقشة وجهات النظر لضمان دقة أكبر، رغم أنه لا يوجد ضمان لصحة التقدير بنسبة 100 %

المشاكل والتحديات:

- سرعة التطور التقني: المشاريع الجديدة غالباً ما تستخدم تقنيات غير مألوفة مما يجعل الخبرة في المشاريع القديمة "عديمة الفائدة" أو غير كافية لتقدير الجهد بدقة.
- الحاجة للتدريب: استخدام أدوات جديدة يتطلب وقتاً إضافياً لتدريب الفريق، وهو عامل قد يغفله المدير عند الاعتماد على خبرته القديمة فقط.

ثانياً: النمذجة الخوارزمية للكلفة (Algorithmic Cost Modeling)

تعتمد هذه التقنية على استخدام منهجية رياضية (معادلات) لحساب الجهد المطلوب بناءً على بيانات كمية وتقنية.

المدخلات الأساسية للمعادلات:

- سمات المنتج (Product Attributes): مثل حجم الكود البرمجي المتوقع أو الوظائف المطلوبة.
- خصائص العملية (Process Characteristics): مثل خبرة الموظفين المشاركين ونموذج التطوير المتبع (Process Model).

أشهر المنهجيات المدرجة تحتها:

- التقدير بنقاط الوظيفة (Function Point Estimation): تقدير الجهد بناءً على عدد الوظائف التي سيقدمها النظام.
- التقدير بحالات الاستخدام (Use Case Estimation): حساب الكلفة بناءً على عدد وتعقيد سيناريوهات الاستخدام المتوقعة.

المميزات: توفر أرقاماً مبنية على معايير ثابتة ومحددة بالموارد اللازمة لكل مهمة (Task) ، مما يقلل من العشوائية في التقدير.

ثانياً: تسعير البرمجيات (Software Pricing)

❖ مفهوم السعر مقابل الكلفة (Price vs Cost):

- **الكلفة (Cost):** هي ما ينفقه المطور فعلياً لإنتاج النظام، وتشمل أجور الموظفين (التي تشكل 70-80٪ من الكلفة)، العتاد، البرمجيات، تكاليف السفر، والتدريب.
- **السعر (Price):** هو المبلغ الذي يطلبه المطور من الزبون، ويساوي عادةً (التكلفة + نسبة الربح + ميزانية الطوارئ) النسبة المتعارف عليها للربح تتراوح بين (40٪ إلى 60٪) فوق التكلفة، لكن هذه النسبة تتغير حسب العوامل التالية.

العوامل المؤثرة في استراتيجية التسعير

1. شروط العقد (Contractual Terms)

يرتبط السعر ارتباطاً وثيقاً بملكية المنتج البرمجي:

- **خفض السعر:** إذا سمح الزبون للمطور بـ الاحتفاظ بحقوق ملكية الكود المصدر (Source Code) ، يمكن للمطور خفض السعر لأنه سيتمكن من إعادة استخدام هذا الكود في مشاريع أخرى مستقبلاً.
- **رفع السعر:** إذا أصر الزبون على امتلاك الكود، يرتفع السعر بنسبة تصل إلى 25-30٪؛ وذلك لتعويض المطور عن فقدان "عقد الصيانة" الاحتكاري الذي كان سيضمن له دخلاً مستقراً ومستداماً.

2. الشك في تقدير الكلفة (Cost Estimate Uncertainty)

بما أن حجم المشروع الحقيقي لا يُعرف بدقة إلا عند انتهائه، فإن تقدير الكلفة يحتوي دائماً على نسبة من الشك.
 إذا كان مدير المشروع غير متأكد من دقة التقديرات (بسبب تقنيات جديدة أو متطلبات مبهمّة)، فإنه يقوم بزيادة السعر لتوفير "هامش طوارئ" (Contingency) يحمي الشركة من الخسارة في حال اضطر الفريق للعمل لساعات إضافية غير محسوبة.

3. فرصة السوق والوضع المالي (Market Opportunity & Financial Health)

تتدخل الأهداف الاستراتيجية للشركة في تحديد السعر:

- **دخول سوق جديد:** قد تقبل الشركة سعراً منخفضاً جداً (حتى لو كان لا يغطي الكلفة) بهدف اكتساح السوق أو الحصول على "Portfolio" قوي في مجال تقني جديد، مما يفتح لها أبواباً لعقود ضخمة مستقبلاً.
- **الوضع المالي:** في الأزمات المالية، قد تضطر الشركات لخفض أسعارها لضمان تدفق السيولة (Cash Flow) وبقاء الموظفين، معتبرة أن الربح القليل أو التعادل (Break-even) أفضل من التوقف عن العمل أو الإفلاس.

4. تغير المتطلبات (Requirements Change)

يستخدم المطورون طبيعة الزبون كأداة للتسعير:

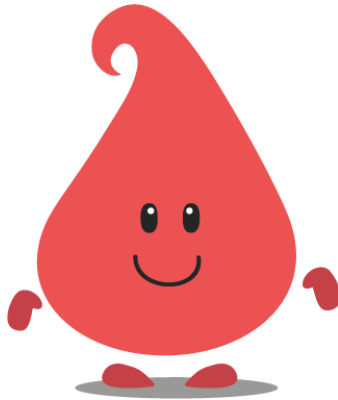
إذا بدا الزبون "متربداً" ومن المتوقع أن يطلب تعديلات كثيرة، قد يقدم المطور سعراً أولاً منخفضاً لربح العقد الثابت، ولكن مع وضع شروط تفرض رسوماً عالية جداً على أي تعديل لاحق للمتطلبات، مما يعوّض النقص في السعر الأصلي.

ثالثاً: استراتيجيات التسعير النهائية

بناءً على ما سبق، تنقسم استراتيجيات الشركات إلى مسارين

1. **سياسة تخفيض السعر (Under-pricing)** : تهدف لربح العقد بأي ثمن، إما للحفاظ على الكوادر البشرية أو لتوسيع النطاق التسويقي.
2. **سياسة تزويد السعر (Increased pricing)** : تتبع عندما يطلب الزبون عقداً بسعر ثابت (Fixed-price contract) ، حيث يرفع المطور السعر لتغطية كافة المخاطر غير المتوقعة التي قد تظهر أثناء التنفيذ.

فائدة: إن نجاح إدارة المشاريع البرمجية يكمن في الموازنة بين دقة التخطيط الهندسي وبين المرونة في التعامل مع المتغيرات، مع وضع "الأشخاص" في قلب العملية الإدارية كأهم مورد استراتيجي.



...انتهت المحاضرة...